Cross – Site – Scripting SEED LAB

Matthew Blanchard

February 18, 2026

Dr. Lyford

CYEN 3331-01

Introduction

Cross-Site Scripting (XSS) remains a critical web security threat, occurring when an application includes untrusted data in a web page without proper validation or escaping. This lab utilizes a containerized Elgg environment to explore the mechanics of Stored XSS, where the malicious payload is permanently stored on the target server. The objective is to demonstrate the progression of an XSS attack from simple alert box execution to sophisticated session-hijacking techniques and self-replicating malware. By simulating both attacker and defender roles, we analyze the bypass of client-side filters and the efficacy of modern browser-based security headers like CSP.

Executive Summary

This report details the systematic exploitation and subsequent defense of a Cross-Site Scripting (XSS) vulnerability within the Elgg social networking platform. By injecting malicious JavaScript into user profile fields, we successfully executed stored XSS attacks to perform unauthorized actions, including automated "Add Friend" requests (GET) and profile data modification (POST). The lab culminated in the development of a self-propagating DOM-based worm that replicated itself across user profiles upon viewing. Finally, the implementation of a Content Security Policy (CSP) demonstrated a robust defense-in-depth strategy, effectively neutralizing the vulnerability by restricting script execution to trusted origins.

Setup

Instructions:

1. Verify that DNS servers are correctly assigned in `/etc/hosts` (Figure 1.1).
2. Bring up docker container in terminal with command `dcup` (Figure 1.2).
    a. The docker container can later be shut down with `^c` where `dcup` was ran or with `dcdown` in another terminal.
3. Run `dockps` in separate terminal to check docker container IDs (This will be needed later) (Figure 1.3).

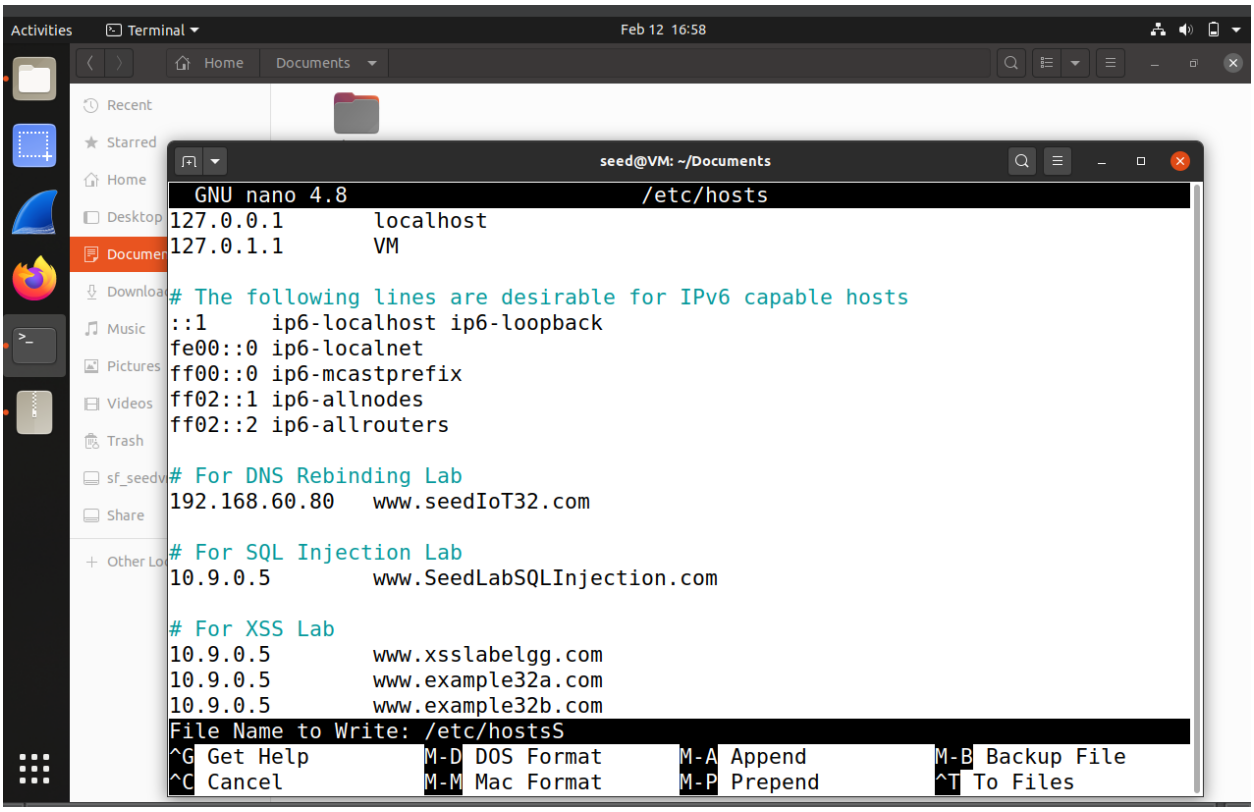

Figure 1.1: DNS assignment.

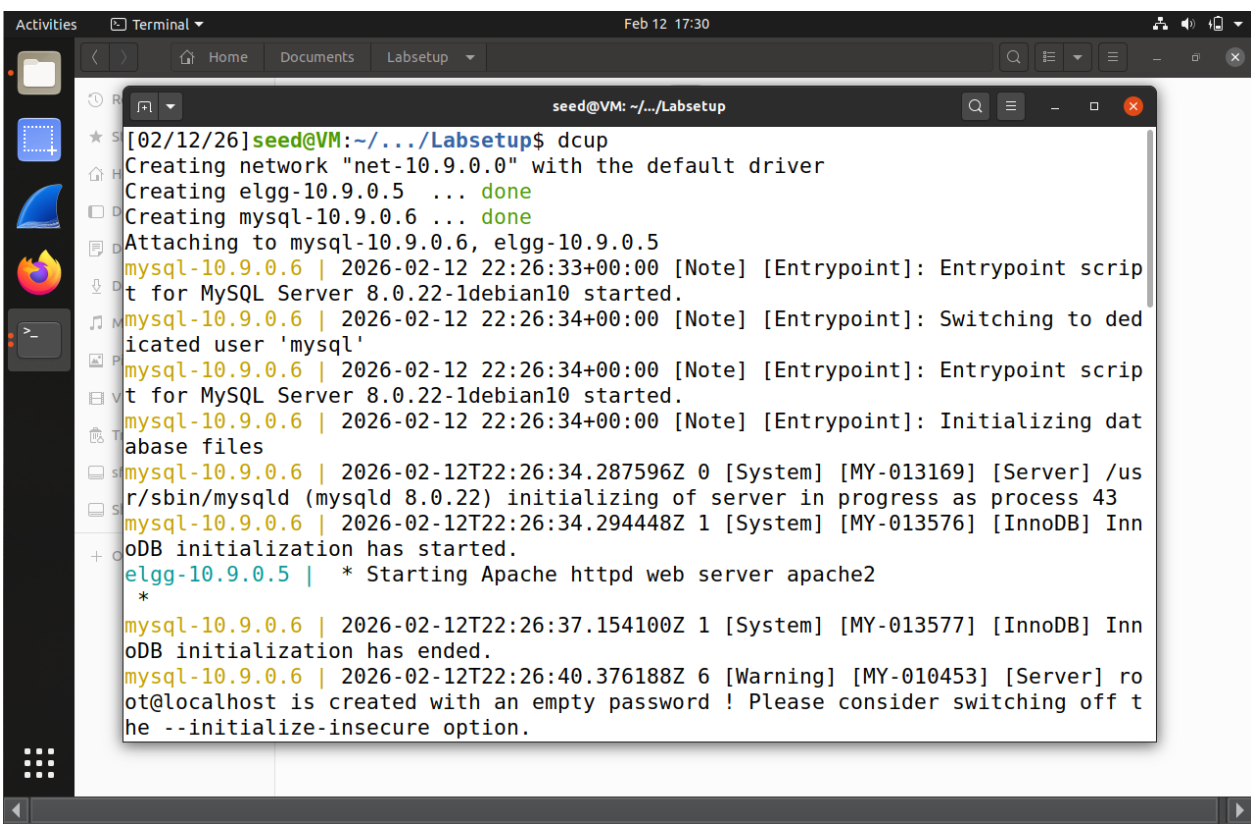

Figure 1.2: Bringing up the docker container.

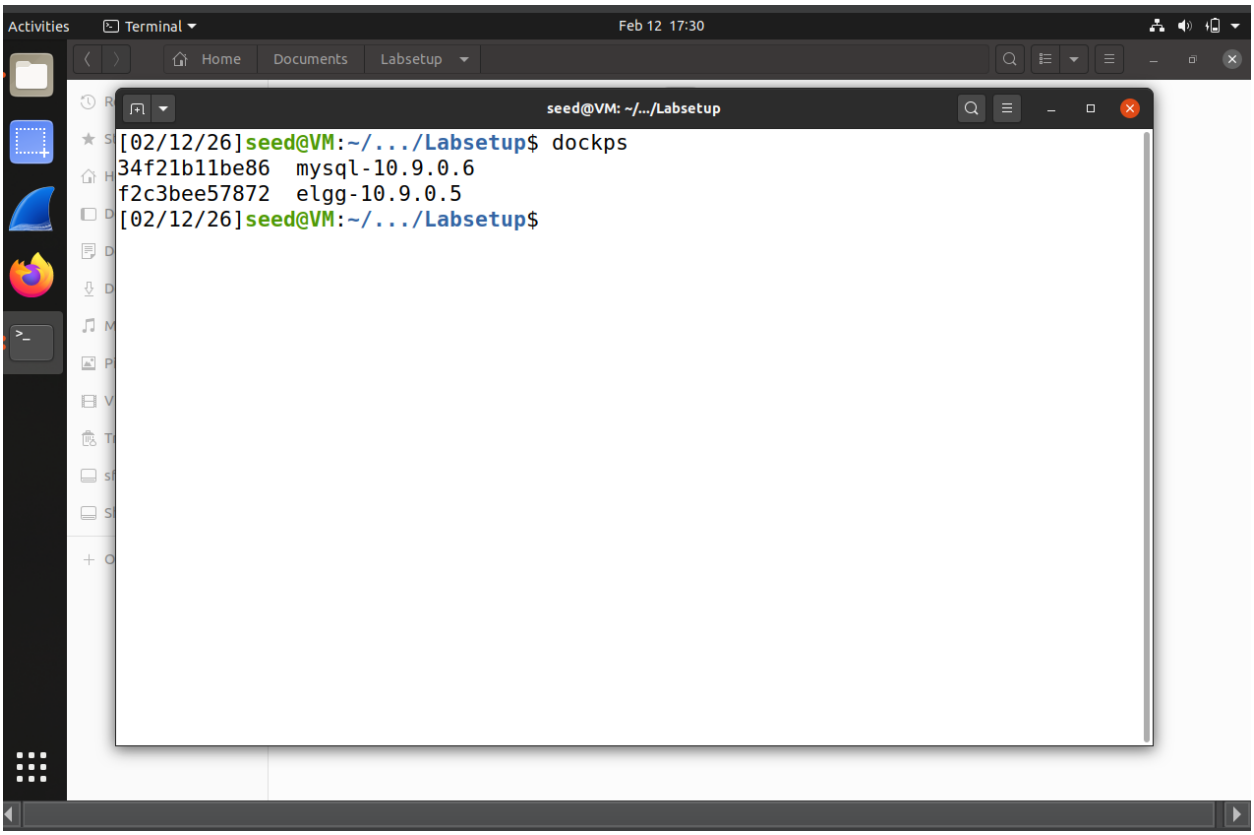

Figure 1.3: Get docker container IDs.

Task 1

Objective:

The objective of this task is to embed a JavaScript program in the attacker's Elgg profile, such that when visited by the victim, the JavaScript program will be executed and an alert window will be displayed.

Instructions:

1. Navigate to webpage, www.seed-server.com.
2. Login as Alice with user, alice, and password, seedalice.
3. Navigate to the profile and edit the profile.
4. In the about me section, click on HTML mode on the top right corner.
5. Inject the javascript code into the About Me field.
6. Save.
7. On the profile page, refresh.

8. Observe.

Observations:

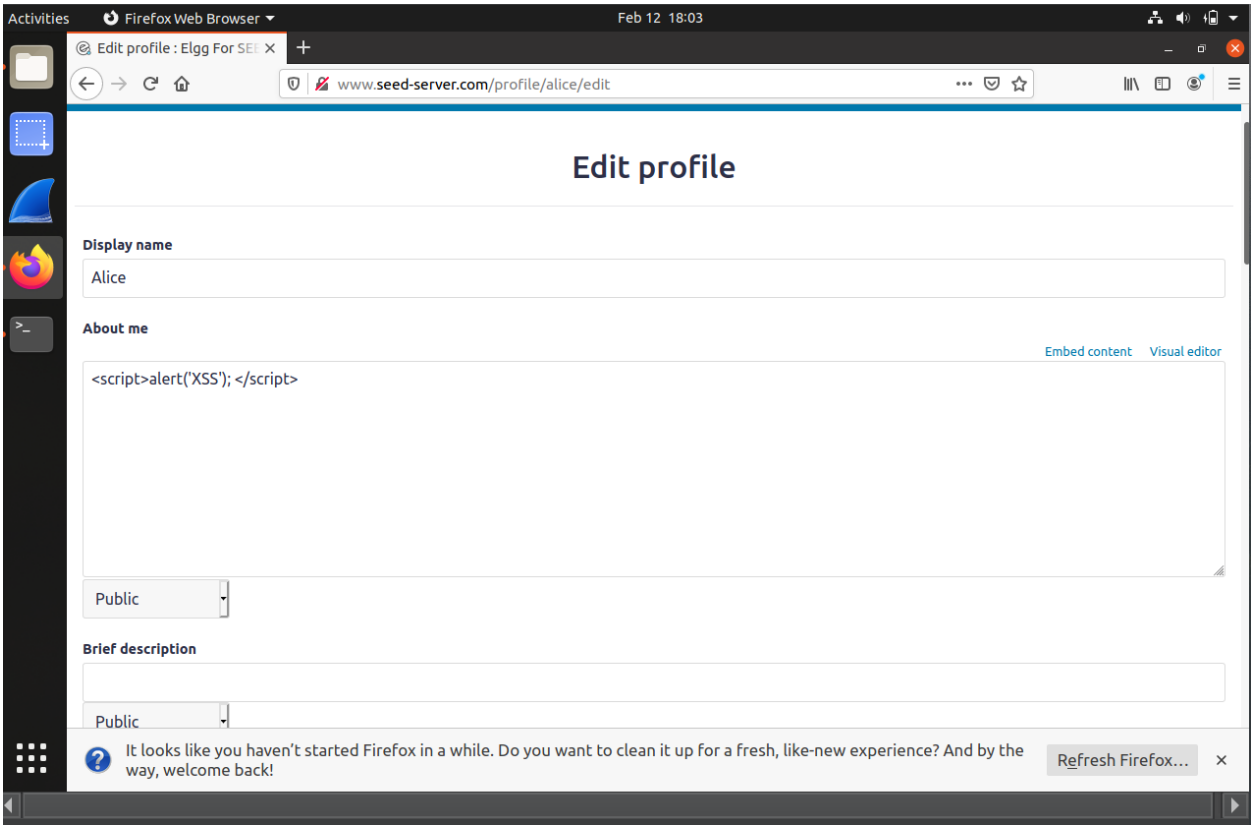

Figure 2.1: Injecting malicious Javascript in the HTML editor.

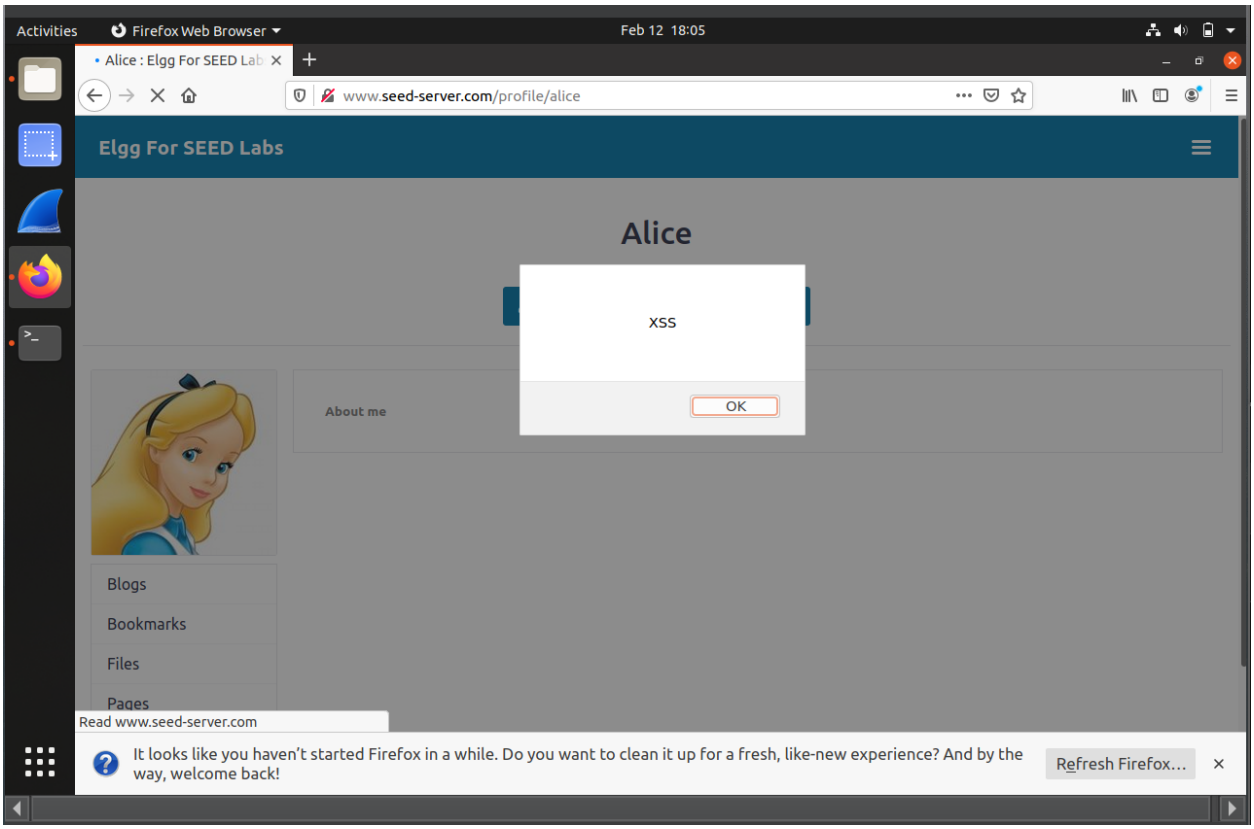

Figure 2.2: Observing the attack.

Looking at the 'About Me' edit page for Alice, it is quite obvious the security vulnerabilities that have been over looked for the sake of the lab. Notably, the 'About Me' input box allows for the direct manipulation of the html code that runs the input box. It is by editing the html directly that the script can be hidden and ran as anything in the 'About Me' field runs to display its contents (Figure 2.1).

Visiting Alice's profile from Boby's profile, the attack is ran and the desired popup is presented, indicating that the script worked as the attacker intended (Figure 2.2).

Task 2

Objective:

The objective of this task is to embed a JavaScript program in the Elgg profile, such that when visited by the victim, the victim's cookies will be displayed in the alert window.

Instructions:

1. Navigate to webpage, www.seed-server.com.
2. Login as Alice with user, alice, and password, seedalice.
3. Navigate to the profile and edit the profile.

4. In the about me section, click on HTML mode on the top right corner.
5. Inject the javascript code into the About Me field.
6. Save.
7. Logout of Alice's account.
8. Login as Boby with user, boby and password, seedboby.
9. Search for Alice's account and visit the profile page.
10. Observe.

Observations:

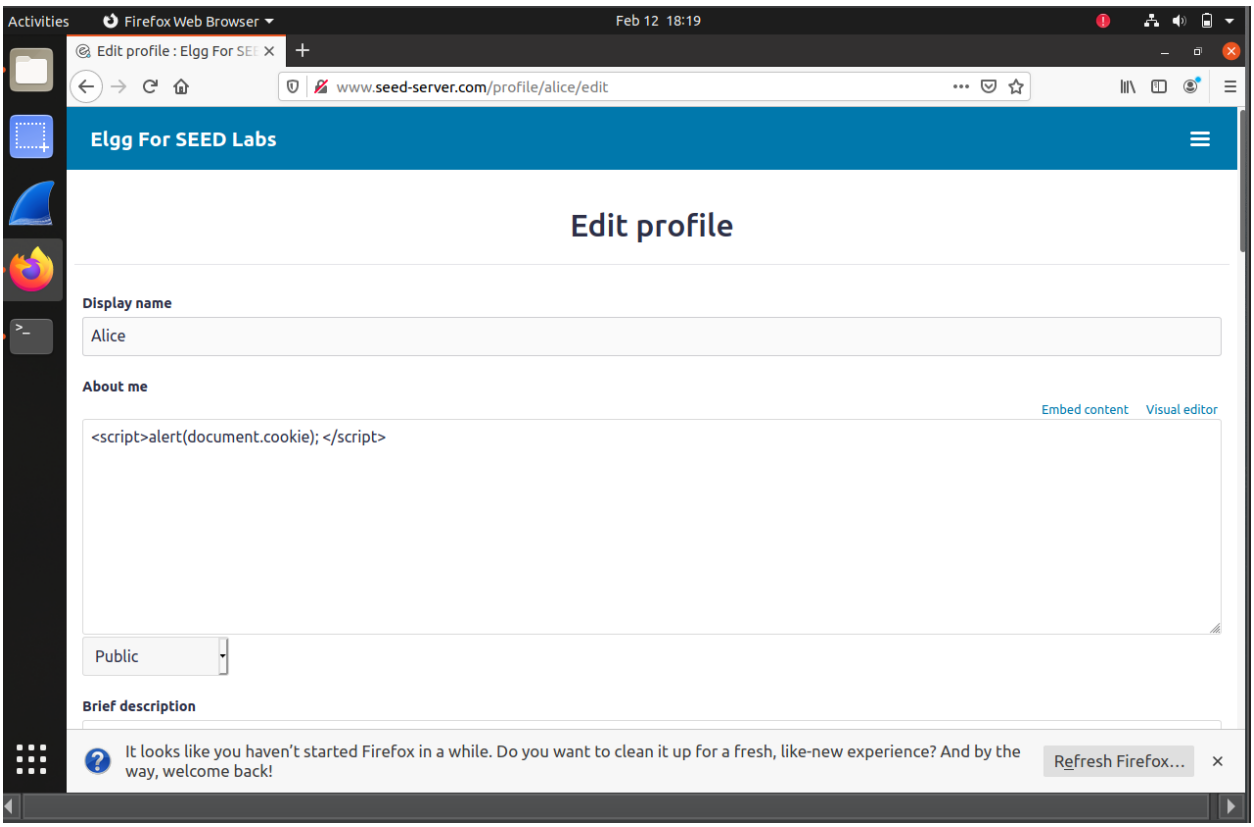

Figure 3.1: Injecting the malicious Javascript code.

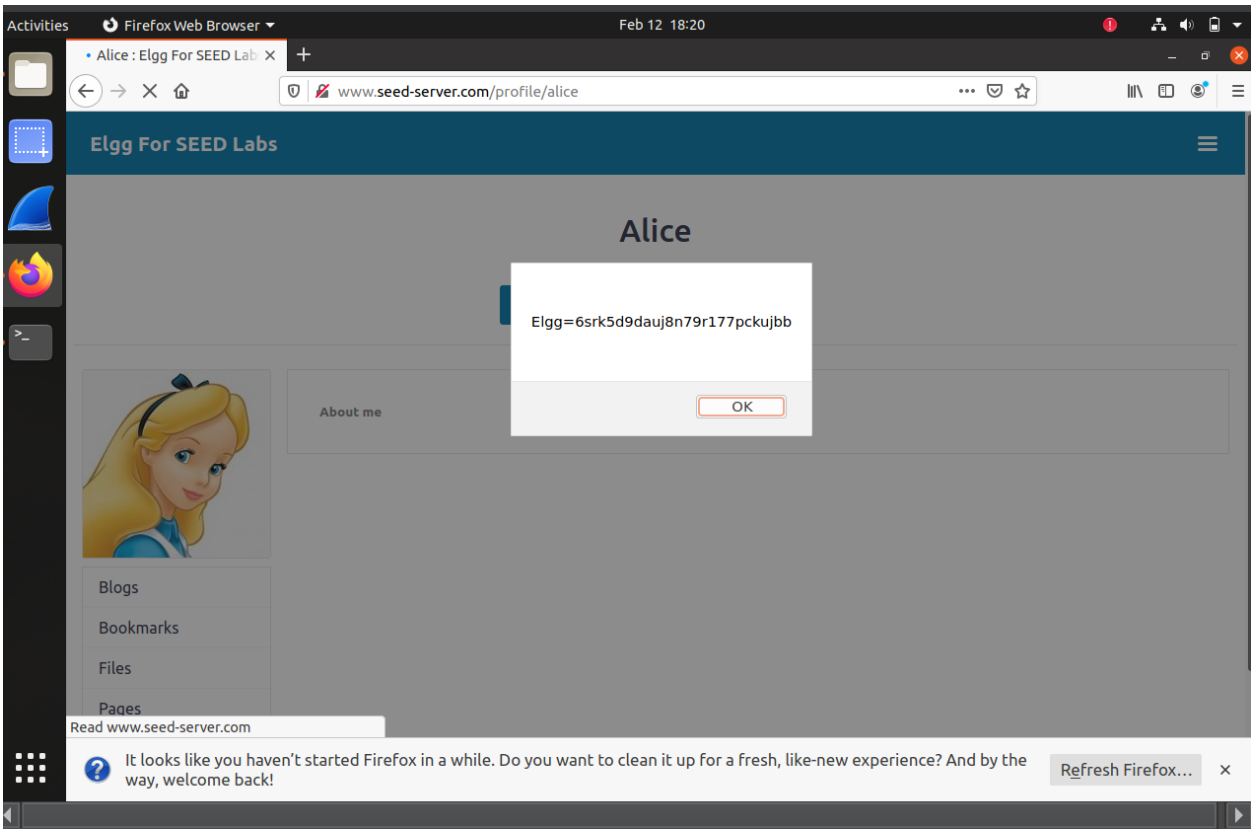

Figure 3.2: Observing the attack.

Using the methods from Task 1, the script to show the victim's cookies in a popup window is injected into Alice's 'About Me' html editor. And as seen in Task 1, this is very simple to do (Figure 3.1). Instead of a string, the cookie is referenced and outputted to the alert window. Logging in as Boby and viewing Alice's profile, Boby's cookies are displayed as intended (Figure 3.2).

Task 3

Objective:

Objective of this task is to have the JavaScript code send the victim's cookies to the attacker. To achieve this, the malicious JavaScript code needs to send an HTTP request to the attacker, with the cookies appended to the request.

Instructions:

1. Navigate to webpage, www.seed-server.com.
2. Login as Alice with user, alice, and password, seedalice.
3. Navigate to the profile and edit the profile.
4. In the about me section, click on HTML mode on the top right corner.

5. Inject the javascript code into the About Me field.
6. Save.
7. Open a terminal window.
8. Type command `nc -lknv 5555` to open and listen on port 5555
9. Logout of Alice's account.
10. Login as Boby with user, boby and password, seedboby.
11. Search for Alice's account and visit the profile page.
12. Observe the terminal output.

Observations:

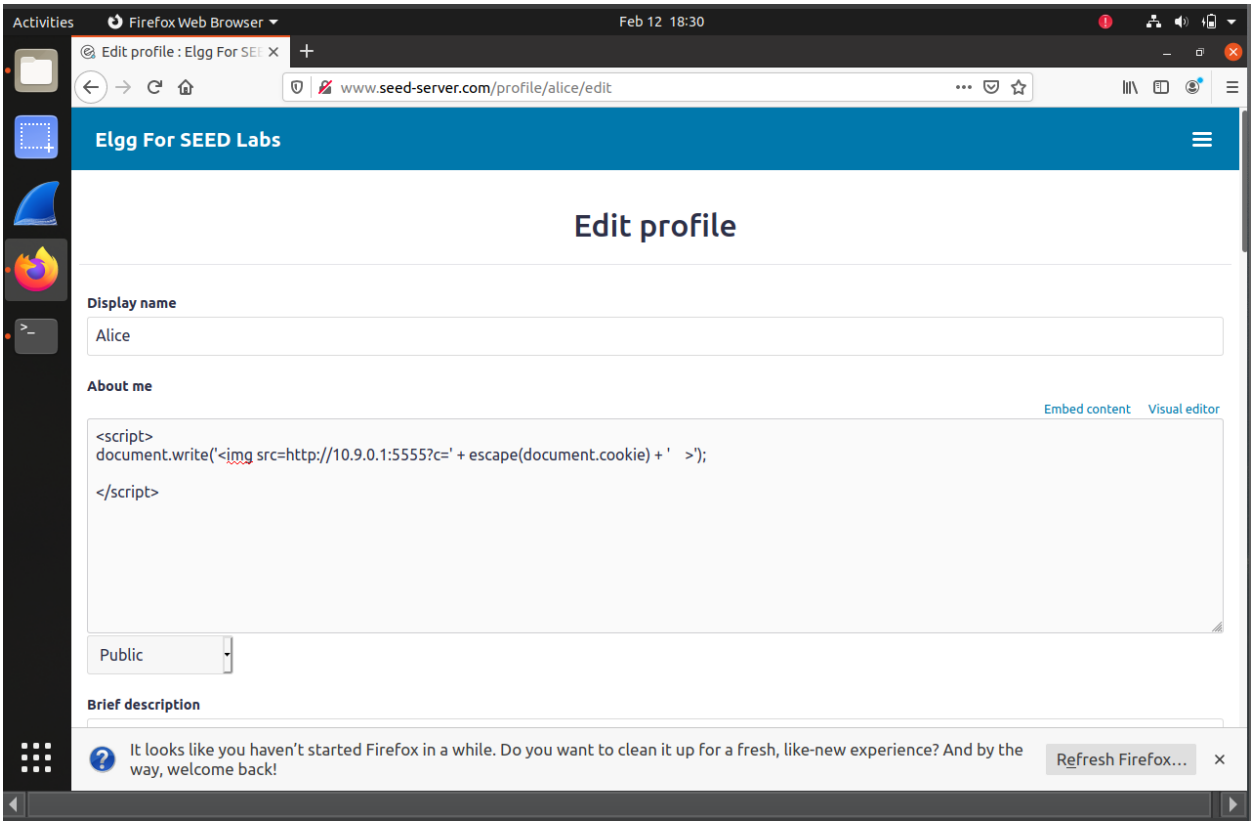

Figure 4.1: Injecting the malicious Javascript code.

Again, the injecting of the malicious js code is very simple and follows the same steps as before but this time, sending the information to the attacker rather than displaying a popup window (Figure 4.1).

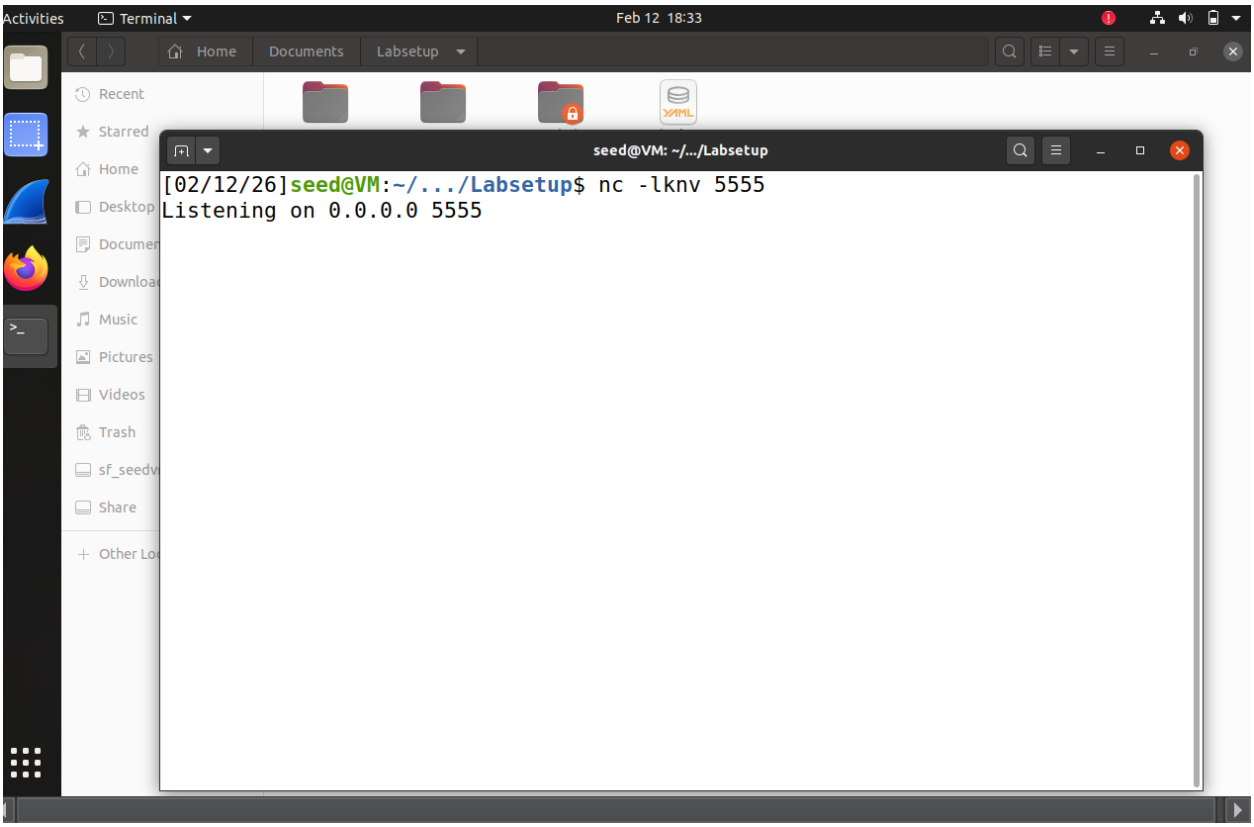

Figure 4.2: Observing the terminal output.

As expected, when visiting Alice's profile as Boby, the HTTP GET request is sent to the attacker via the opened port (port 5555), which the attacker has set up to listen for incoming request. This way, the attacker can easily gain sensitive information such as the victim's cookies sent straight to the attacker's machine.

Task 4

Objective:

The objective of this task is to write a malicious JavaScript program that forges HTTP requests directly from the victim's browser, without the intervention of the attacker. The objective of the attack is to add the attacker as a friend to the victim

Instructions:

1) Navigate to webpage, www.seed-server.com.
2) Login as Samy with user, samy, and password, seedsamy.
3) Press F12, go to the console page and enter the command `elgg.session.user.guid` to get Samy's ID.
4) Navigate to the profile and edit the profile.

5) In the about me section, click on HTML mode on the top right corner.
6) Inject the javascript code into the About Me field using the discovered guid.
7) Save.
8) Logout of the account.
9) Login as Alice with user, alice, and password, seedalice.
10) Search for Samy's profile and visit the profile page.
11) Check Alice's Friends page.
12) Observe the page.

Observations:

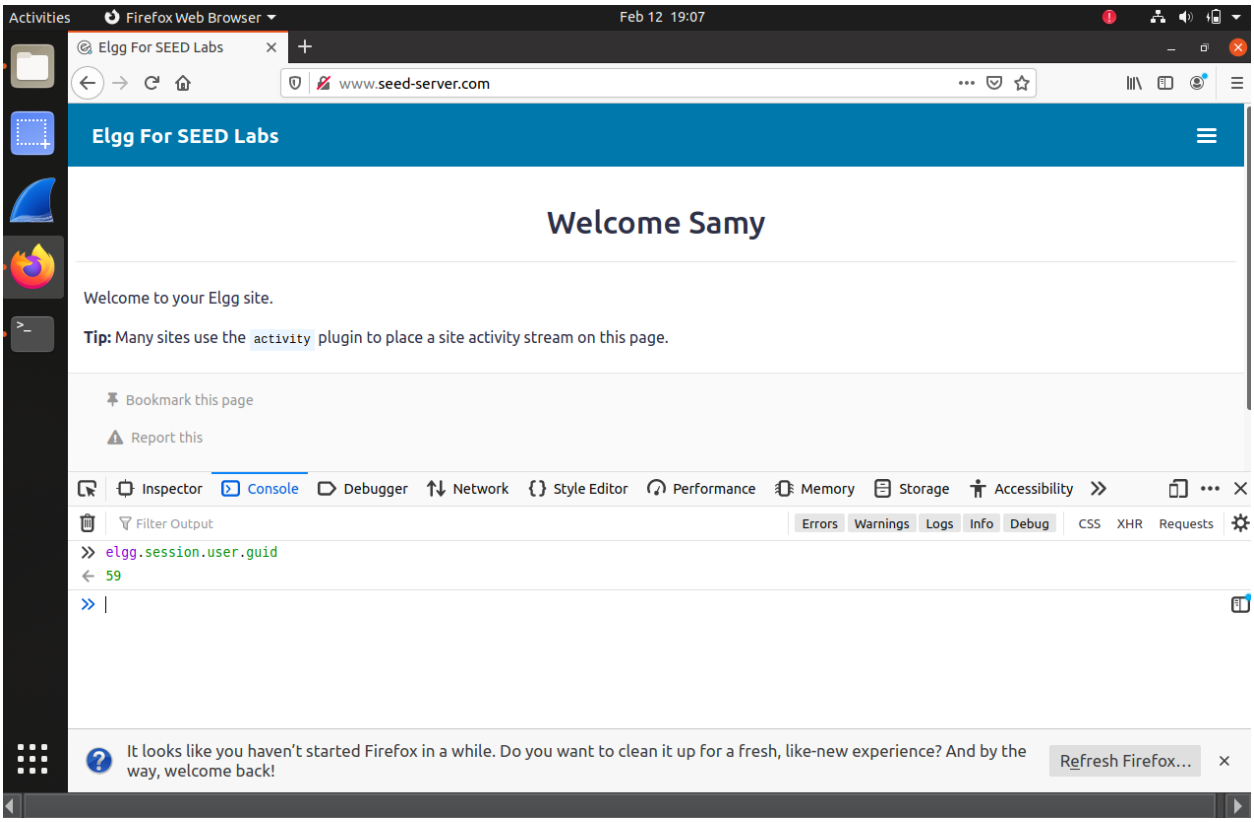

Figure 5.1: Getting Samy's guid.

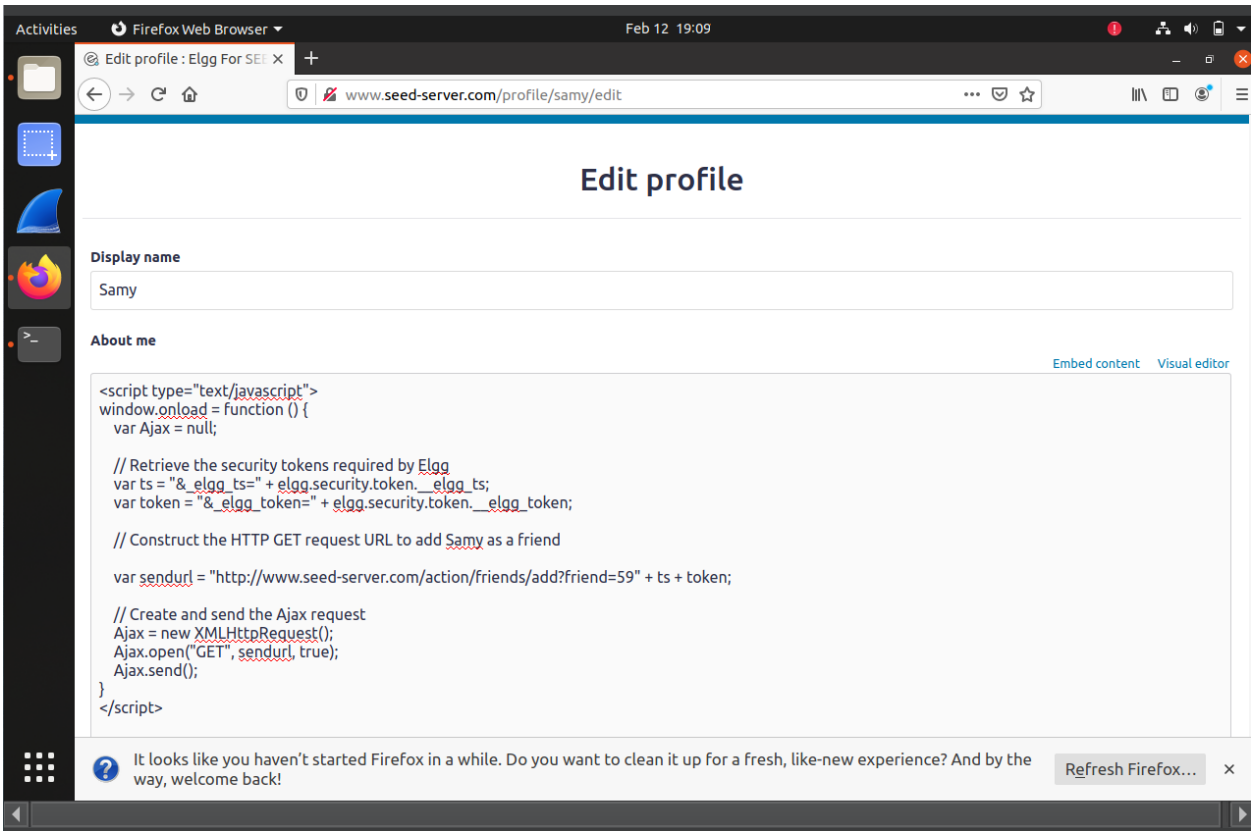

Figure 5.2: Injecting the malicious Javascript code.

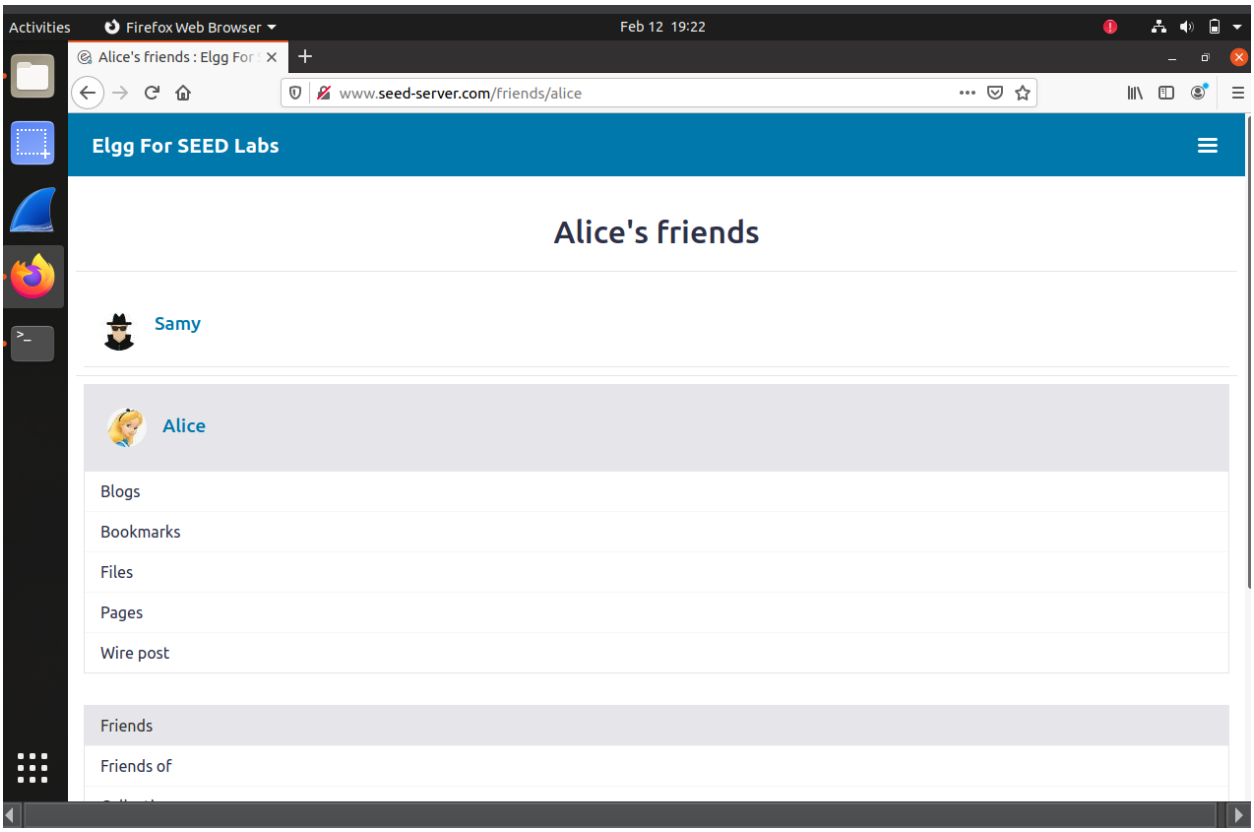

Figure 5.3: Observing the results.

Like in previous tasks, the JS code embedded into Samy's About Me page triggers when visited. As Alice, visiting Samy's About Me page triggers the JS code and adds Samy to Alice's friends list as expected (Figures 5.1-3). These lines, 'var ts=' and 'var token=', retrieve the session-specific timestamp and security token used by Elgg to prevent Cross-Site Request Forgery (CSRF) attacks. They are necessary because the server will reject any "Add Friend" request that does not include these valid, matching tokens. If the Elgg application only provided the Editor mode, it would become much harder to perform this attack as the JS code would automatically have html tags inserted like <p> and <br> that break JS syntax.

Task 5

Objective:

The objective of this task is to modify the victim's profile when the victim visits Samy's page. Specifically, modify the victim's "About Me" field.

Instructions:

1) Navigate to webpage, www.seed-server.com.
2) Login as Samy with user, samy, and password, seedsamy.
3) Navigate to the profile and edit the profile.
4) In the about me section, click on HTML mode on the top right corner.
5) Inject the javascript code into the About Me field.
6) Save.
7) Logout of the account.
8) Login as Alice with user, alice, and password, seedalice.
9) Search for Samy's profile and visit the profile page.
10) Check Alice's profile page.
11) Observe the About Me field.
12)

Observations:

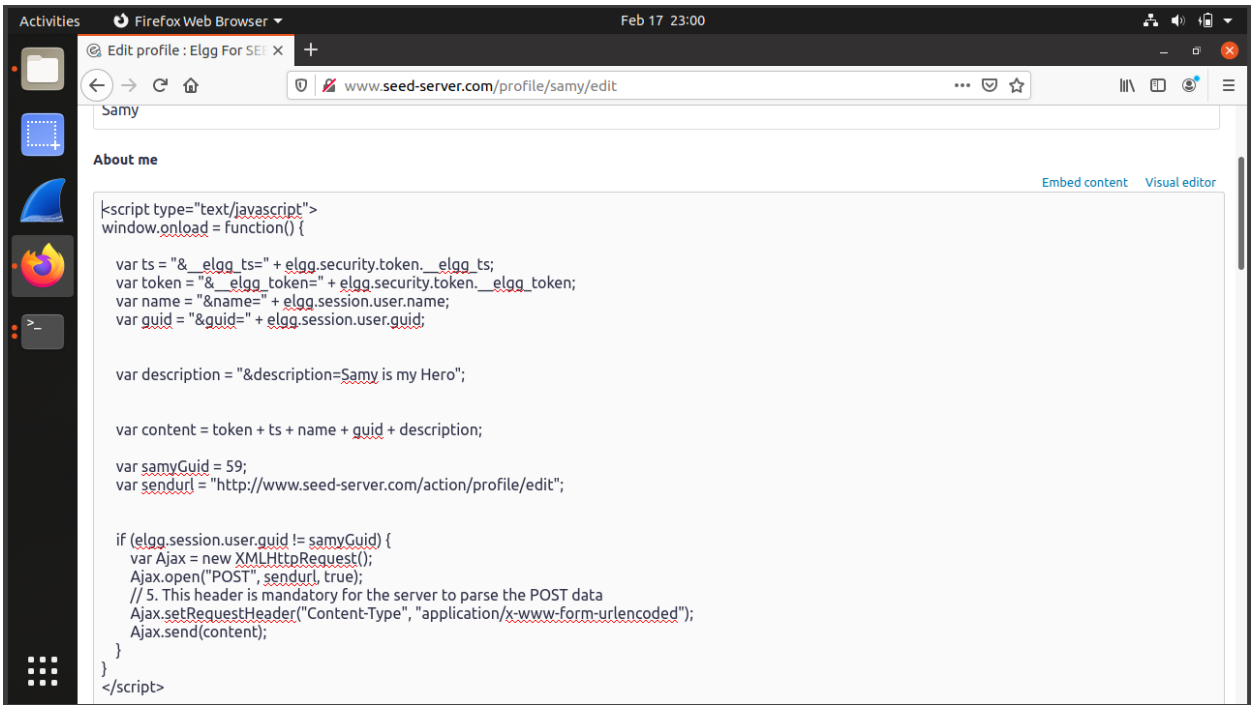

Figure 6.1: Injecting the malicious Javascript code.

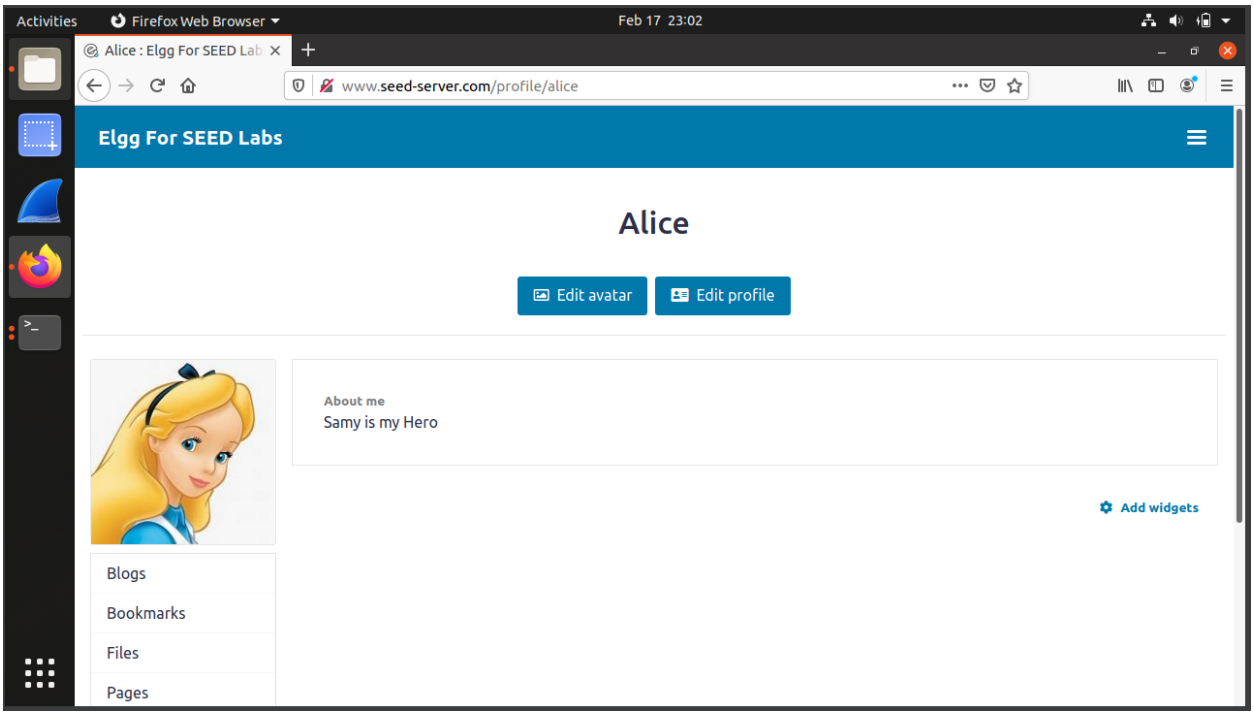

Figure 6.2: Observeing the attack.

Similar to the previous task, when Sammy's About Me page is visited by another user, Alice, their About Me page is edited to the attacker's message (Figures 6.1-2).

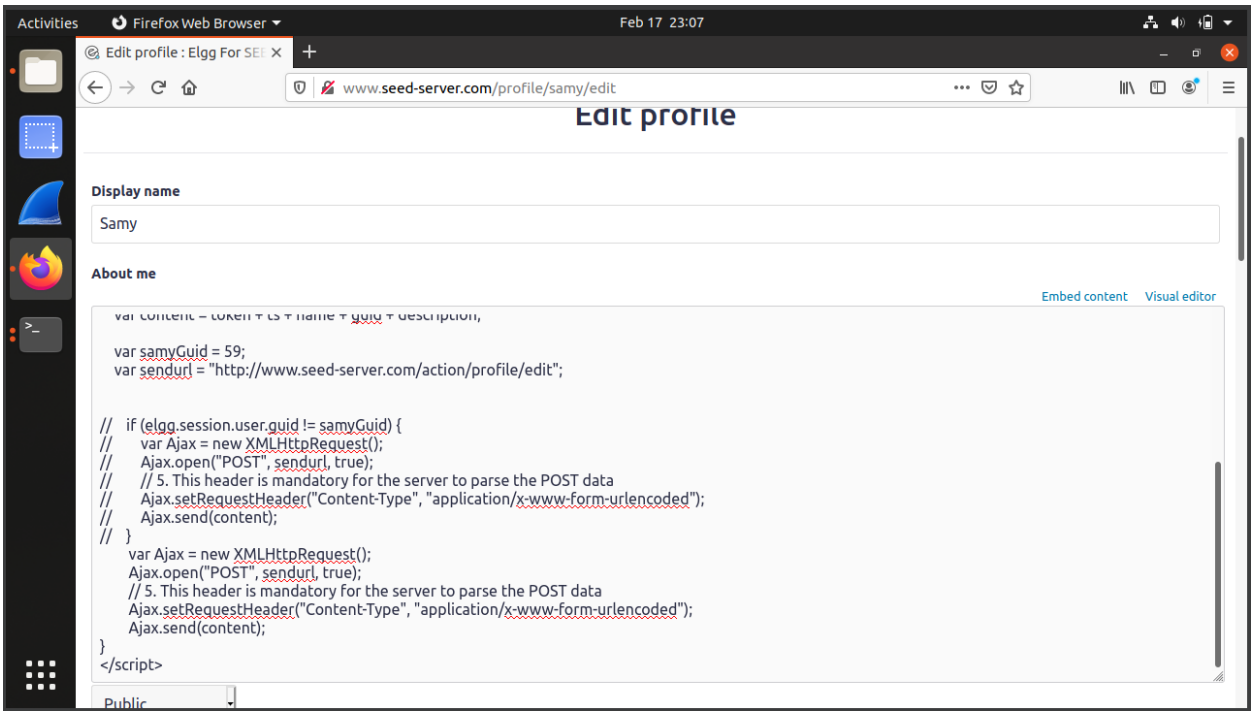

Figure 6.3: Commenting out the guid check.

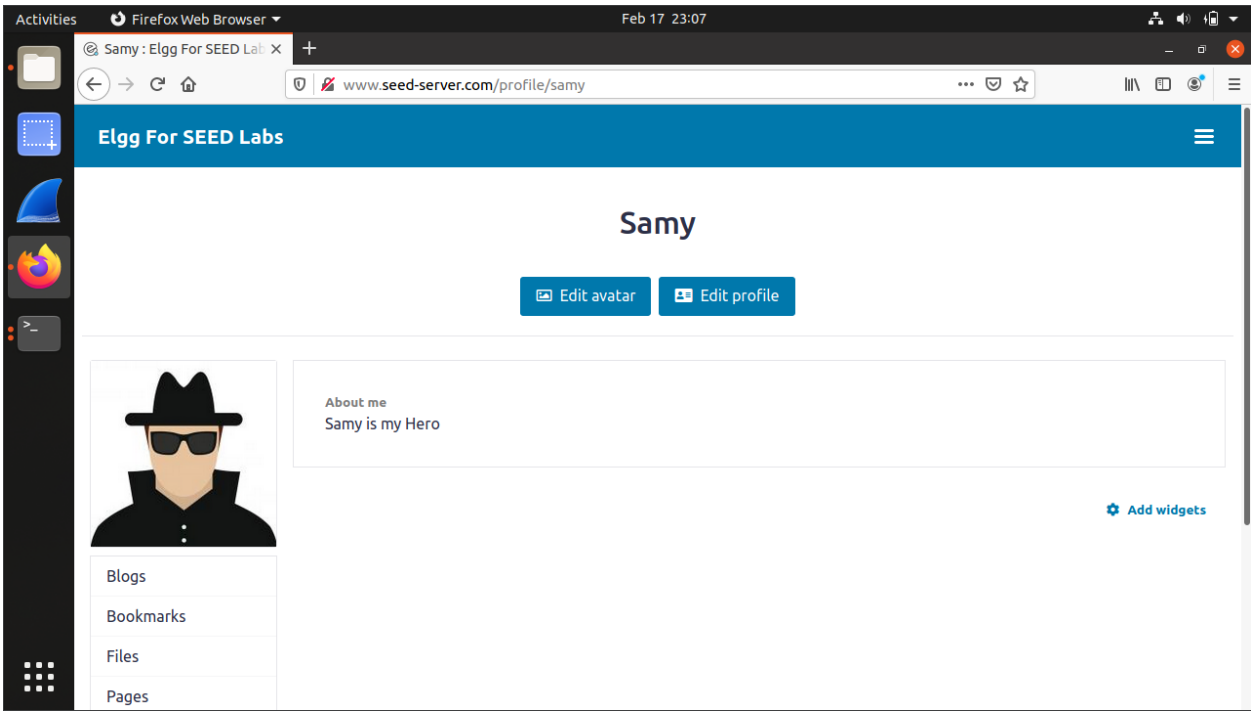

Figure 6.4: Observing the self-attack.

Without the check that prevents the JS code from attacking Samy, when reloaded, Samy's About Me page changes and the attack message overwrites the JS code, essentially deleting the attack entirely (Figures 6.4-5).

Task 6

Objective:

The objective of this task is to transform the script into a self-propagating worm that, when viewed, automatically copies its own malicious code into the victim's profile so it can continue spreading to every subsequent user who views the newly infected profile.

Instructions:

1) Navigate to webpage, www.seed-server.com.
2) Login as Samy with user, samy, and password, seedsamy.
3) Navigate to the profile and edit the profile.
4) In the about me section, click on HTML mode on the top right corner.
5) Inject the javascript code into the About Me field.
6) Save.
7) Logout of the account.
8) Login as Alice with user, alice, and password, seedalice.
9) Clear the About Me field from previous task.
10) Search for Samy's profile and visit the profile page.
11) Check Alice's profile.
12) Edit the profile.
13) Switch to HTML mode.
14) Observe the About Me field.

Observations:

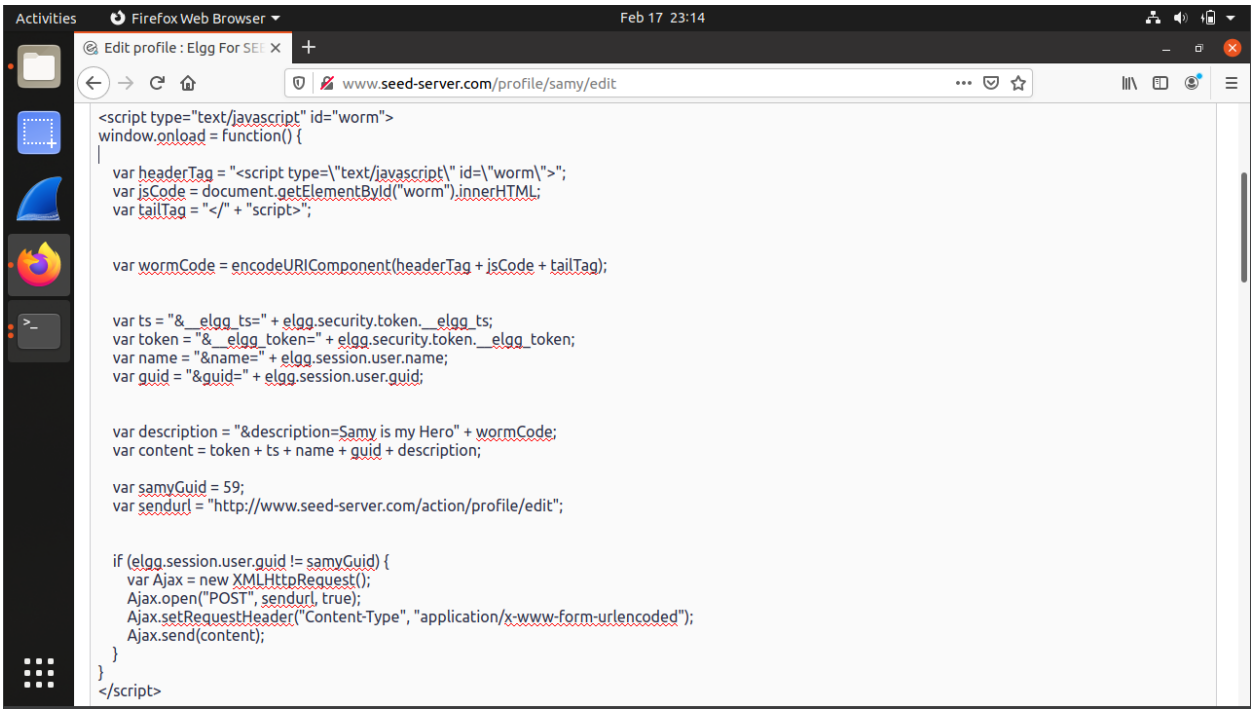

Figure 7.1: Injecting the malicious Javascript code.

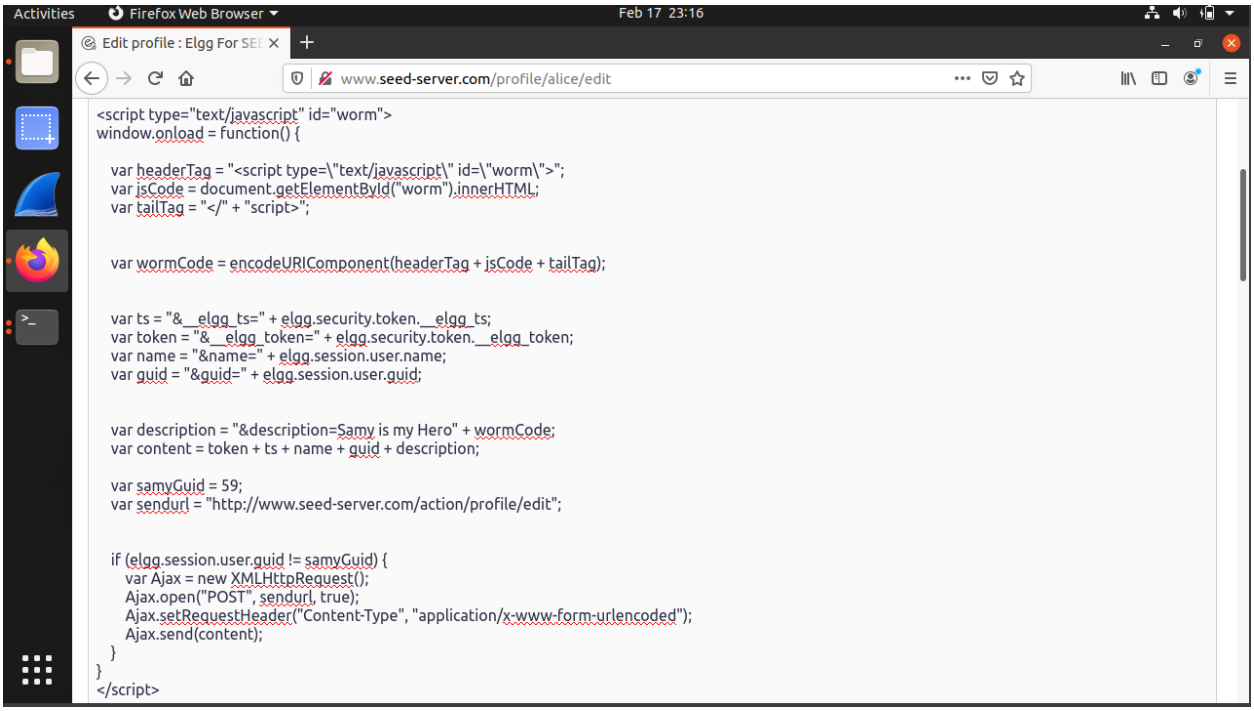

Figure 7.2: Observing the replicated code.

This time, with the JS code now having the ability to self-replicate, when visited by the victim, Alice, there is no obvious evidence that Alice's About Me page has changed. However, going into Editor mode on Alice's page, the JS code from Samy has 'wormed' its way into Alice's About Me page indicating the worm is functioning as intended (Figures 7.1-2).

Task 7

Objective:

The objective of this task is to observe how a Content Security Policy (CSP) can be used as a powerful defense mechanism to block XSS attacks even when a vulnerability exists in the application code. In this task, you enable a CSP header and witness how it prevents your previous scripts from executing by strictly controlling which scripts the browser is allowed to run.

Instructions:

1. Visit each site, example32a.com, example32b.com, example32c.com.
2. Observe each website and if the button works or not.
3. Open a new terminal window.
4. Run `dockps` and note the elgg ID.
5. Run `docker exec -it ID /bin/bash`.
    a. Replace ID with the docker elgg ID.
6. Inside the docker, run `nano /etc/apache2/sites-enabled/apache_csp.conf`.
7. Observe the file contents.
8. Navigate to configuration settings for example32b.com
9. Add in source for area 5.
    a. This can be found in viewing the source code of the web page.
10. Write out with `^O` and `ENTER` then exit with `^X`.
11. Restart the Apache service with `service apache2 restart` in the docker terminal.
12. Go back to example32b.com page and refresh.
13. Observe the changes.
14. Locate phpindex.php in the lab's root directery in the csp folder.
15. Add in sources for areas 2 and 5.
    a. This can be found in viewing the source code for example32c.com
16. Save the file.
17. Enter the terminal running in the docker container and run `cd /var/www/csp`.
    a. Enter `ls` command to ensure phpindex.php is present in this directory.

18. Highlight the hostname with the working directory of the docker terminal.
    a. It will look like `docker_id:/var/www/csp` where `docker_id` will be the docker container ID found after running `dockerps`.
19. From a terminal window in the edited phpindex.php directory, run docker `cp phpindex.php docker_id:/var/www/csp/` to copy the edited php file into the php file in the running docker.
20. In the docker terminal, run `service apache2 restart`.
21. Go to the webpage example32c.com and refresh the site.
22. Observe the changes.

Observations:

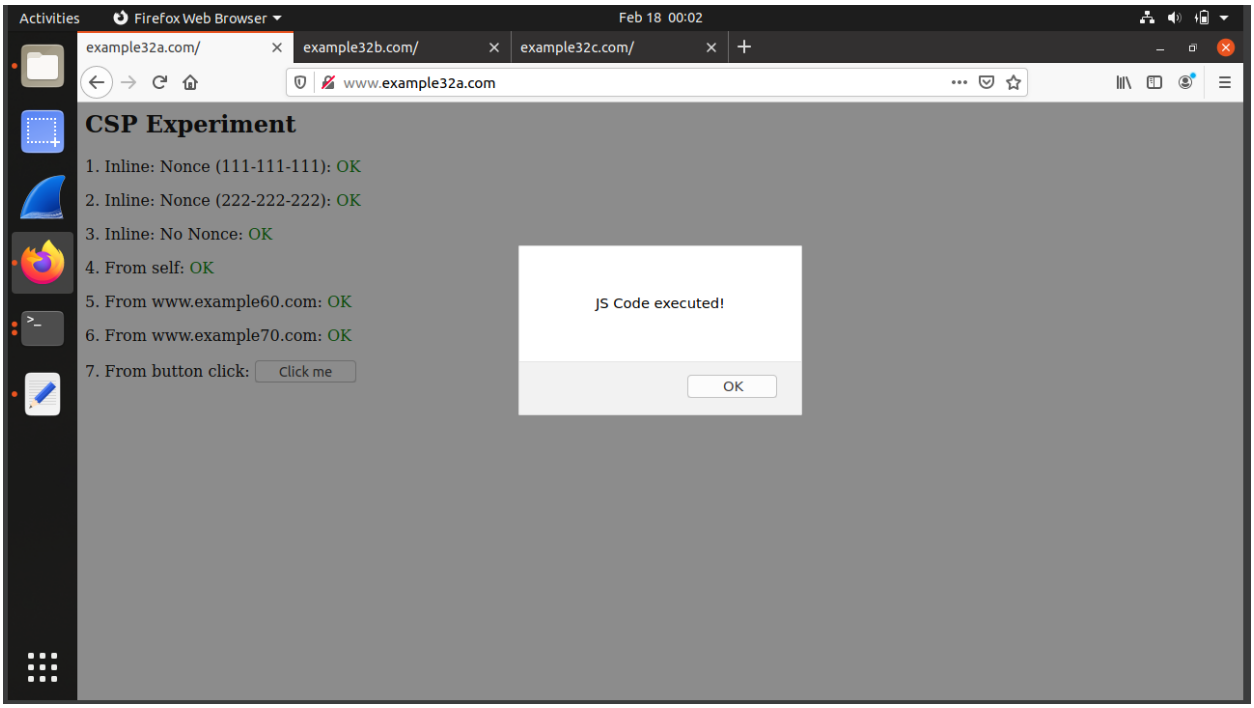

Figure 8.1: Default configuration of example32a.com.

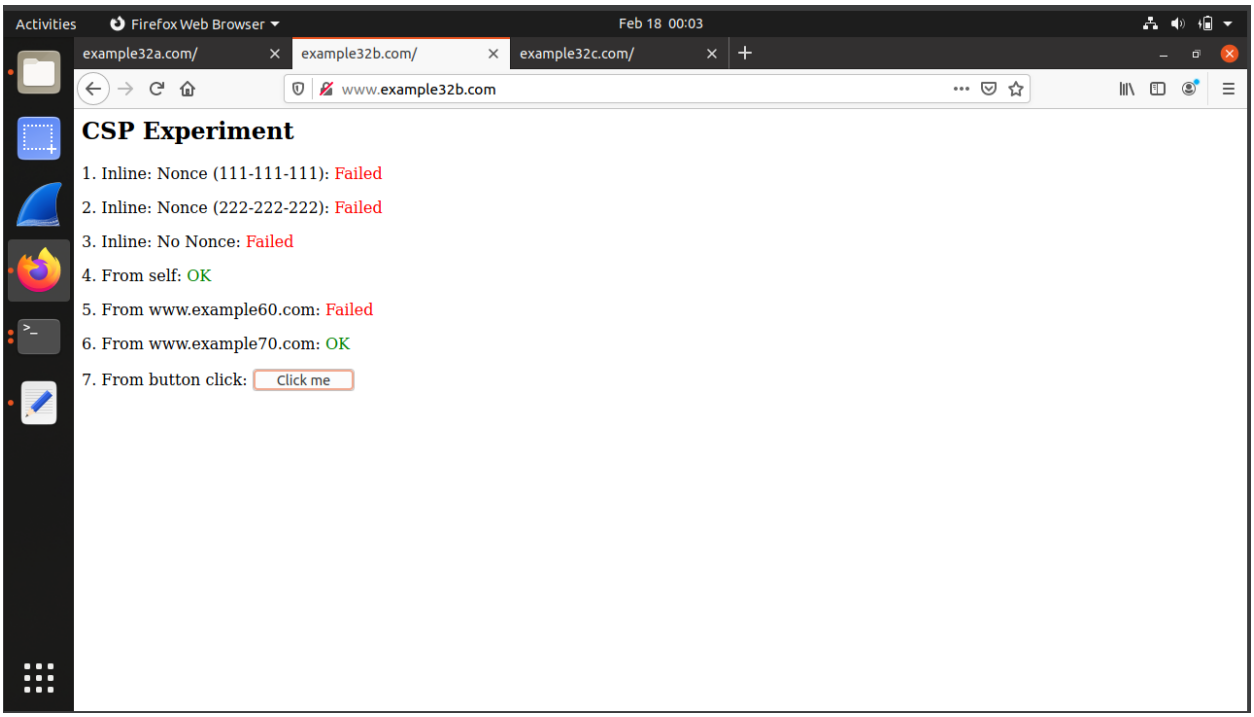

Figure 8.2: Default configuration of example32b.com.

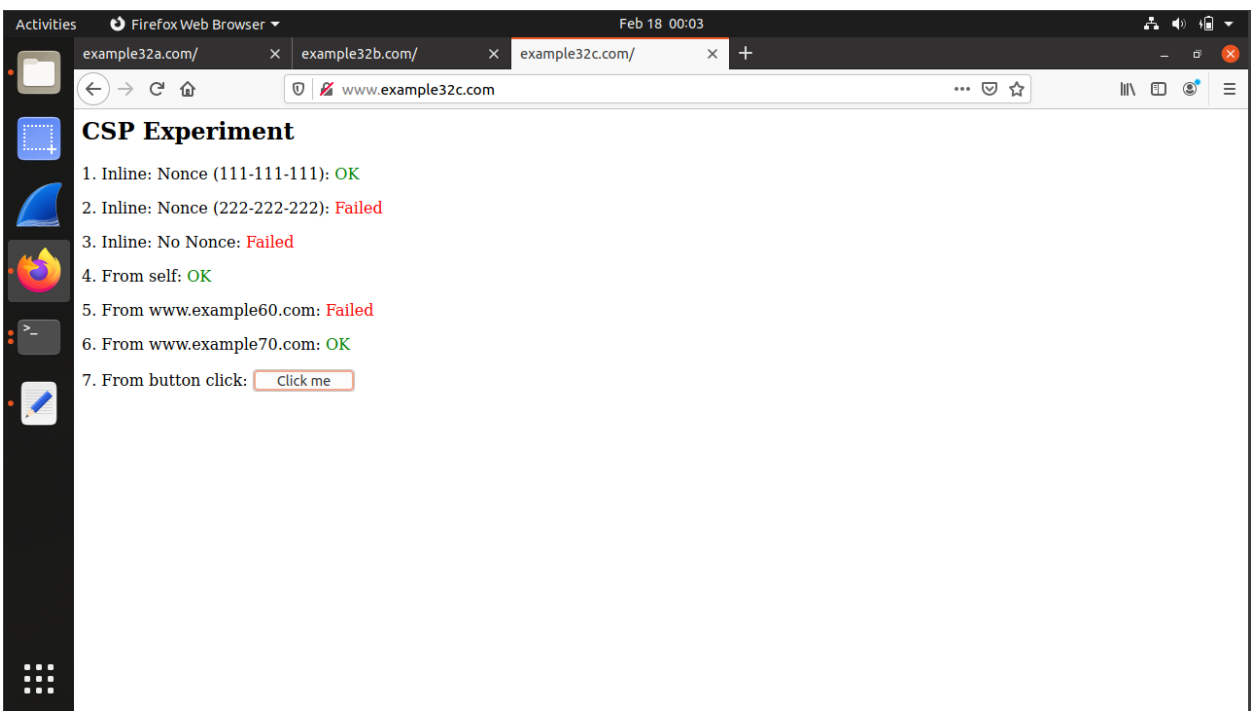

Figure 8.3: Default configuration of example 32c.com.

Without editing any configuration files, visiting each of the example sites shows the default configuration state of those sites. The site, example32a.com, has all lines enabled signifying that JS is enabled from all sources including those outside of its own site (Figure 8.1). The site, example32b.com, has all but areas 4 and 6 disabled, indicating only some JS sources are allowed to run (Figure 8.2). The same observation can be made with example32c.com with areas 1, 4, and 6 enabled (Figure 8.3). Additionally, only the button in example32a.com worked.

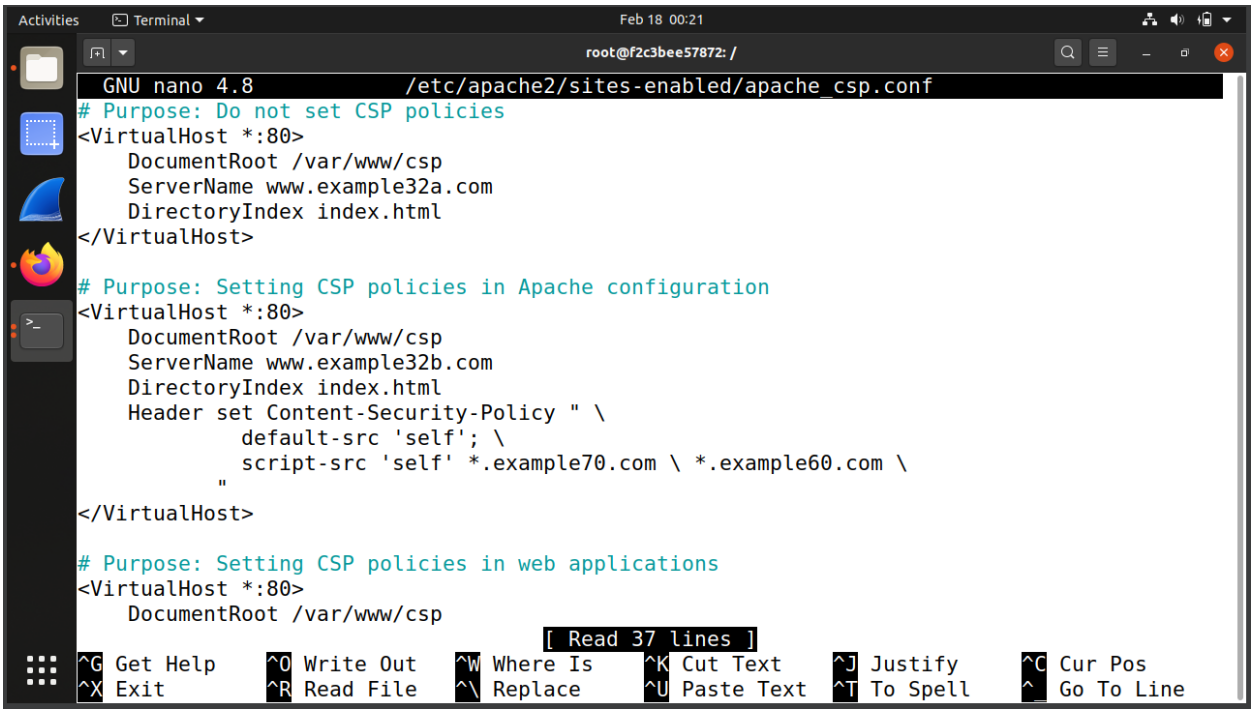

Figure 8.4: Editing the Apache configuration file for example32b.com.

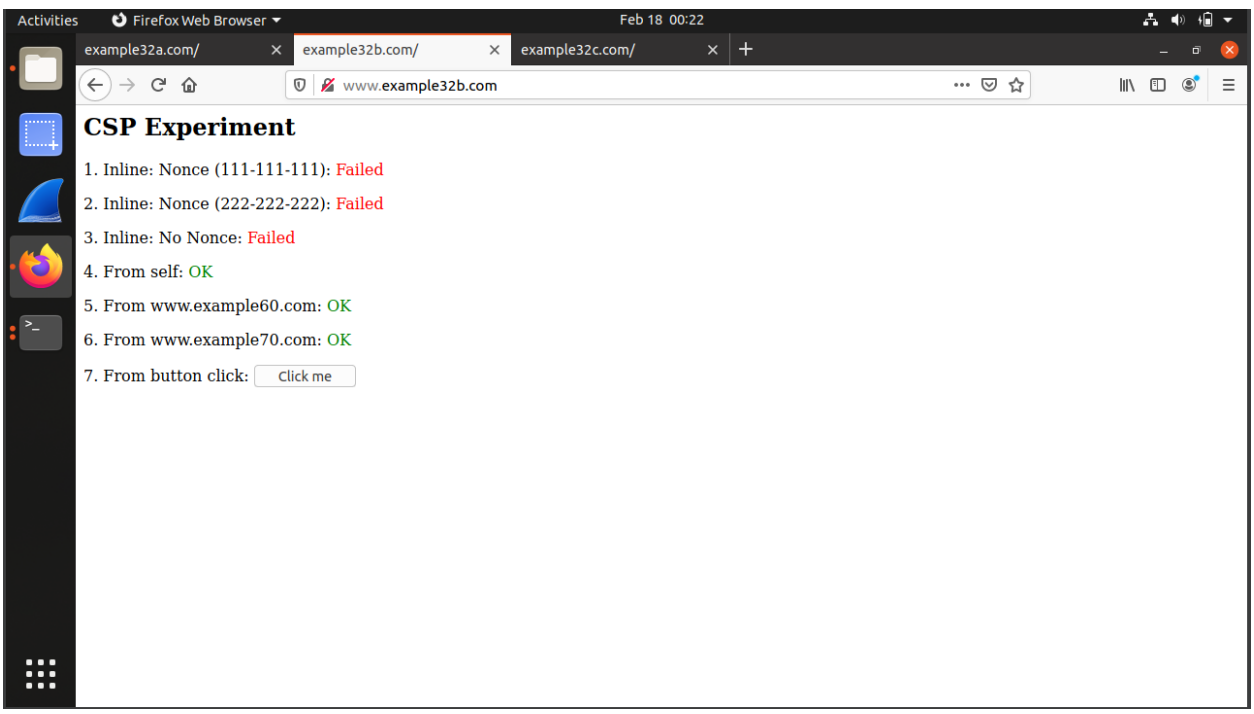

Figure 8.5: Observing the effects of the edited configuration.

Navigating to the Apache configuration file within the docker allows for the configuration file to be modified without restarting the entire docker container. Only the Apache service has to be restarted. From here, the CSP of example32b.com can be edited to have area 5 enabled by replicating the syntax for area 6 (Figure 8.4). Doing this and restarting the Apache service, the change to the CSP is immediate on site refresh. Now on example32b.com, area 5 is enabled with the others (Figure 8.5).

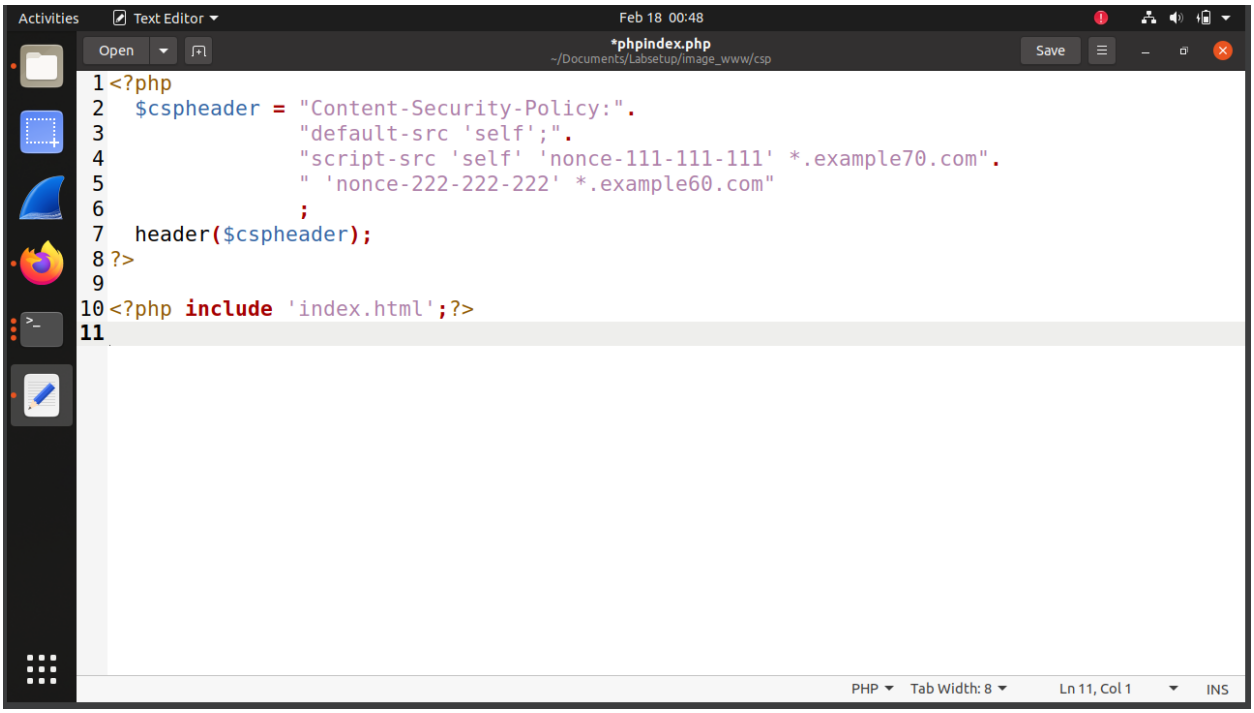

Figure 8.6: Editing the php file for example32c.com.

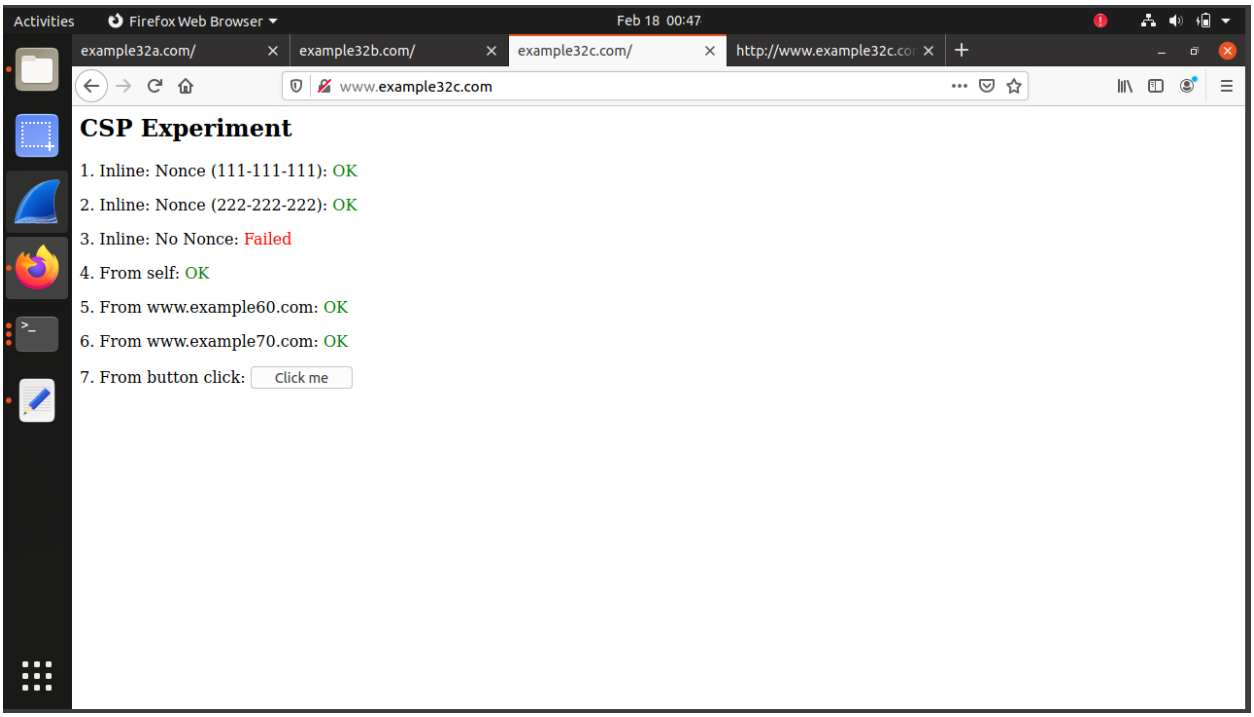

Figure 8.7: Observing the effects of the edited configuration.

The same method can be used to change the CSP configuration of example32c.com to enable areas 2 and 5. The only difference is that the DirectoryIndex is set to an external php file located outside of the Apache configuration file. Editing this file is simpler as the Apache service does not need to be restarted. To do this, navigate to phpindex.php outside of the docker, edit the CSP header to include more areas, saving, and copying the php file to the php file inside of the

docker (Figure 8.6). Then, reloading the web page, areas 1, 2, 4, 5, and 6 will be enabled (Figure 8.7).

Content Security Policies can greatly reduce and eliminate vulnerabilities to cross-site-scripting attacks by only allowing JS code from pre-approved sources to run. This means any injected code injected into any HTML tags will be rejected as the site will not load any code that is not specified as a source in the CSP.

Conclusion

The experiments confirm that reliance on input sanitization alone is often insufficient, especially when administrative configurations can be bypassed to allow raw HTML input. The successful propagation of the DOM-based worm highlights the danger of "wormable" vulnerabilities in social ecosystems, where a single infection can lead to a platform-wide compromise.

However, the transition to Task 7 proved that Content Security Policy (CSP) is a highly effective mitigation. Even while the underlying vulnerability (the lack of input filtering) remained, the CSP directive script-src 'self' prevented the browser from executing the injected code, thereby protecting the user. This underscores the necessity of a multi-layered security architecture that combines server-side validation with proactive browser-level security policies.